

Failure (prove the suite catches a regression) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
baseline_score = run_eval_suite()
print("baseline:", baseline_score)

# manually edit the prompt file: remove "always mention the return policy"

sabotaged_score = run_eval_suite()
print("after sabotage:", sabotaged_score)

if sabotaged_score < baseline_score:
    print("Suite works: it caught the regression.")
else:
    print("Suite has a hole: score didn't drop. Go tighten the rules for this beh

# restore the original prompt file before continuing
```

This is a correct, minimal version of the check. It reports the result but doesn't tell you *which* task caught (or missed) the problem.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — compare overall score only

```
def run_regression_check() -> None:
    baseline = run_eval_suite()
    print(f"baseline: {baseline.overall_score} ({baseline.passed}/{baseline.total}")

    input("Now remove one real instruction from the prompt, save, then press Enter")

    sabotaged = run_eval_suite()
    print(f"after sabotage: {sabotaged.overall_score} ({sabotaged.passed}/{sabotaged.total}")

    if sabotaged.overall_score < baseline.overall_score:
        print("PASS: the suite caught the regression.")
    else:
        print("FAIL: the suite did NOT catch this regression. Rules need tightening")

if __name__ == "__main__":
    run_regression_check()
```

Approach 2 — compare per-task results, to see exactly which test caught it

```
def run_regression_check() -> None:
    baseline = run_eval_suite() # returns a dict: {task_id: passed(bool)}

    input("Now remove one real instruction from the prompt, save, then press Enter")

    sabotaged = run_eval_suite()

    newly_failed = [
        task_id
        for task_id in baseline
        if baseline[task_id] and not sabotaged[task_id]
    ]

    print(f"baseline passed: {sum(baseline.values())}/{len(baseline)}")
    print(f"after sabotage passed: {sum(sabotaged.values())}/{len(sabotaged)}")
```

```
if newly_failed:
    print(f"PASS: suite caught it. Newly failing task(s): {newly_failed}")
else:
    print("FAIL: no task newly failed. Find which task SHOULD cover this inst

if __name__ == "__main__":
    run_regression_check()
```

Difference from Basic: Basic tells you *that* something changed. Approach 2 tells you *which specific task* caught it — which is exactly what you need when the check fails: it points you straight at the rule that needs tightening, instead of leaving you to guess across your whole suite. Neither Intermediate approach yet asks whether a single before/after pair of numbers is even reliable evidence — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a noise floor from repeated baseline runs, plus per-task diffing

```
def run_n_baselines(n: int = 3) -> list[dict]:
    return [run_eval_suite() for _ in range(n)] # each returns {task_id: passed(

def run_regression_check_with_noise_floor() -> None:
    baseline_runs = run_n_baselines(3)
    baseline_scores = [sum(r.values()) / len(r) for r in baseline_runs]
    noise_floor = max(baseline_scores) - min(baseline_scores)
    print(f"baseline scores across 3 runs: {baseline_scores} (noise floor: {noise

    # decide the bar BEFORE sabotaging, so the result can't be rationalized after
    minimum_real_drop = noise_floor + 0.10

    input("Now remove one real instruction from the prompt, save, then press Enter")

    sabotaged = run_eval_suite()
    sabotaged_score = sum(sabotaged.values()) / len(sabotaged)
    average_baseline = sum(baseline_scores) / len(baseline_scores)
    drop = average_baseline - sabotaged_score

    # use the LAST baseline run for a clean per-task diff (same conditions as the
    last_baseline = baseline_runs[-1]
    newly_failed = [
        task_id for task_id in last_baseline
        if last_baseline[task_id] and not sabotaged[task_id]
    ]

    print(f"average baseline: {average_baseline:.2f}, after sabotage: {sabotaged_
    print(f"minimum drop to count as real: {minimum_real_drop:.2f}")

    if drop > minimum_real_drop and newly_failed:
        print(f"PASS: suite caught a real regression. Newly failing: {newly_faile
    elif drop > 0 and drop <= minimum_real_drop:
        print("INCONCLUSIVE: drop is within normal run-to-run noise. Not strong e
    else:
        print("FAIL: no meaningful drop. Rules need tightening.")
```

```
if __name__ == "__main__":
    run_regression_check_with_noise_floor()
```

Expected output (illustrative, real numbers vary by run):

```
baseline scores across 3 runs: [0.9, 0.8, 0.9] (noise floor: 0.10)
Now remove one real instruction from the prompt, save, then press Enter...
average baseline: 0.87, after sabotage: 0.60, drop: 0.27
minimum drop to count as real: 0.20
PASS: suite caught a real regression. Newly failing: ['t3', 't7']
```

If instead `drop` had come back at `0.05` — smaller than the `0.10` noise floor already measured with *no* prompt change — the Intermediate version above would have reported it as a straightforward "the suite caught it," when really that 0.05 could easily just be ordinary variance. This version correctly reports it as `INCONCLUSIVE` instead.

Difference from Intermediate, and what this adds: Intermediate compares exactly one baseline run against one sabotaged run and trusts the gap. This Approach runs the baseline 3 times first specifically to measure the suite's own run-to-run wobble with the prompt *unchanged* — that wobble becomes the noise floor a real regression has to clearly exceed. It also decides the pass bar (`minimum_real_drop`) before sabotaging anything, so the result can't be rationalized after seeing it, and reuses the per-task diffing from Intermediate Approach 2 on top of the noise-floor check, rather than replacing it — the two checks answer different questions (is the drop real at all, and if so, which specific task caught it).

Which one should you actually use? For a suite that's fully deterministic (temperature=0 everywhere, or no LLM calls at all in the scored path), Intermediate Approach 2's single before/after comparison is already reliable — there's no run-to-run wobble to worry about, so a noise floor adds cost for no benefit. The moment any part of your scored path — the pipeline, or the judge — has real sampling variance, use this Advanced version: 3 baseline runs is a small, worthwhile cost against the real risk of this document's whole Failure exercise reporting a false "PASS: the suite works" that's actually just noise you never measured.